Project Overview and Evaluation

Project overview

ComRes is a community equipment-sharing application for Community Resource Network SA. Individuals can browse listed equipment, check availability, and request a paid hire, temporary free loan, or permanent donation. Approved community organisations can maintain their own listings and handle requests for those items. Administrators manage organisation applications, inventory, requests, and fulfillment records.

The application is implemented with Next.js 16, React 19, TypeScript, Tailwind CSS 4, Neon PostgreSQL, Stripe, and Vercel Blob. See the technical handover for scope boundaries and the developer setup guide for operating instructions.

Scope and deliverables

The project deliverables include:

1. Public equipment catalogue, search, item details, availability, and request form.
2. Requester registration/login, request history, cancellations, feedback, and reviews.
3. Organisation registration, administrator moderation, provider-owned equipment listings, and request handling.
4. Administrator inventory, blackout, organisation, request, payment, and fulfillment management.
5. PostgreSQL schema, application source, tests, documentation, and setup/maintenance scripts.

For the detailed deliverables index and listed source paths, see ComRes Deliverables and Scope Guide. Source paths shown beside links in the matrix identify locations inside the packaged source archive and can also be used in the full GitHub repository.

Evaluation evidence matrix

Status meanings: *Pass* means the specific automated test run passed; *Partial* means some code/test/design evidence exists but the full criterion has not been demonstrated; *Not tested* means no execution evidence was found. Implemented code or a planned procedure is not a test result.

#	Evaluation criterion and requirement	Implemented feature / design	Test and source references	Available evidence and status	Outstanding work / limitations
1	**Scope completion** — verify the agreed features and end-to-end workflows.	Catalogue, roles, requests, organisation/provider flows, administration, fulfillment, and payment workflows are represented in the app.	Source areas: `app/`, `components/`, `lib/`, schema (source path: `db/schema.sql`); scope guide.	Documentation and source inventory exist. Automated suite ran locally. **Partial.**	Complete a scope walkthrough on the team's deployment and record results against the agreed requirements.
2	**Availability and booking** — check equipment availability, quantity, and conflicts.	Date-range availability, booking/blackout constraints, inventory quantities, and booking request validation.	availability.ts (source path: `lib/availability.ts`), availability.test.mjs (source path: `lib/availability.test.mjs`), request API (source path: `app/api/requests/route.ts`), availability calendar (source path: `components/AvailabilityCalendar.tsx`).	Availability suite passed as part of RUN-2026-11-02-01. **Partial** for end-to-end behavior.	Capture user-visible calendar and conflicting/over-quantity request results using approved deployed test data.
3	**Listing management** — verify provider listing, image, availability, and administrator moderation.	Approved organisations can list and edit their items; administrators manage inventory and organisation status.	provider items (source path: `app/org/items/page.tsx`), provider item API (source path: `app/api/org/items/route.ts`), image upload API (source path: `app/api/uploads/route.ts`), admin inventory (source path: `app/admin/items/page.tsx`), admin organisations (source path: `app/admin/organisations/page.tsx`); organisation design.	Feature/source references and a planned walkthrough exist. Organisation response/query unit tests passed but do not test listing CRUD. **Not tested** manually.	Demonstrate provider approval, listing create/edit, image upload, availability/blackout, and admin moderation; record sanitized evidence.
4	**Transaction workflows** — verify paid hire, temporary free loan, and permanent donation.	These transaction types are represented in item/request and fulfillment logic; payments are associated with approved bookings where configured.	transaction types (source path: `lib/types.ts`), booking/payment rules (source path: `lib/booking-payment.ts`), booking/payment tests (source path: `lib/booking-payment.test.mjs`), fulfillment logic (source path: `lib/fulfillment.ts`), fulfillment tests (source path: `lib/fulfillment.test.mjs`), request API (source path: `app/api/requests/route.ts`).	Related fee/payment/fulfillment unit tests passed locally. **Partial.**	Complete an end-to-end request and fulfillment per transaction type on the approved test environment.
5	**Cancellation and returns** — verify cancellation, collection, return, and condition records.	Eligible requests can be cancelled; approved hire/loan bookings record collection and return inspection; donations record handover; condition and notes are stored.	cancellation tests (source path: `lib/cancellation.test.mjs`), availability tests (source path: `lib/availability.test.mjs`), fulfillment tests (source path: `lib/fulfillment.test.mjs`),	Relevant cancellation, availability, and fulfillment tests passed locally. **Partial.**	Record a deployed cancellation, collection, return inspection, condition, and resulting availability/payment state.

#	Evaluation criterion and requirement	Implemented feature / design	Test and source references	Available evidence and status	Outstanding work / limitations
			fulfillment actions (source path: `components/FulfillmentActions.tsx`), fulfillment API (source path: `app/api/bookings/[id]/fulfillment/route.ts`).		
6	**Payment demonstration** — demonstrate Stripe test payments, deposits, and refunds without real funds.	Stripe test-mode booking payments include hire-fee/deposit records, saved payment method, webhook-confirmed status, retry, and eligible refund flows. See Technical Notes for the current behavior and legacy-route caveat.	payment workflow (source path: `lib/booking-payment-workflow.ts`), payment setup tests (source path: `lib/payment-setup.test.mjs`), idempotency tests (source path: `lib/payment-idempotency.test.mjs`), Stripe webhook (source path: `app/api/stripe/webhook/route.ts`), refund API (source path: `app/api/admin/requests/[id]/payments/refund/route.ts`).	Payment logic/response tests passed locally; no Stripe event or payment demonstration was recorded. **Partial** (logic tests only).	Demonstrate authorized Stripe test-mode success/failure/refund and webhook state using team-provided test accounts; preserve event references without credentials or card data.
7	**Administrative reporting** — review listings, requests, transaction status, and availability.	Administrator dashboard displays pending requests, upcoming pickups, active items, pending organisations; admin pages cover inventory, requests, organisations, feedback, and availability management.	admin dashboard (source path: `app/admin/page.tsx`), admin requests (source path: `app/admin/requests/page.tsx`), admin items (source path: `app/admin/items/page.tsx`), admin organisations (source path: `app/admin/organisations/page.tsx`).	Source and implementation references only; no captured admin reporting results. **Not tested.**	Demonstrate these views and an allowed test-data management action as an authorized administrator.
8	**Usability** — observe key workflows and responsive layout; collect feedback.	Responsive page/component layouts exist; a timestamp formatting test is not a usability assessment.	UI areas: `app/`, `components/`; timestamp test (source path: `lib/format-timestamp.test.mjs`); manual checklist.	No usability session, observation, responsive test, or user feedback record. **Not tested.**	Conduct and document an approved user walkthrough at desktop and narrow/mobile sizes; obtain consent for any captured feedback/images.
9	**Database quality** — verify required fields, relationships, validation, duplicate prevention.	Schema defines constraints, references, unique user email/review-per-request, booking/fulfillment records, and transaction/status checks.	schema (source path: `db/schema.sql`), schema setup tests (source path: `scripts/apply-schema.test.mjs`), migration safety tests (source path: `scripts/payment-migration-safety.test.mjs`), SQL parser tests (source path: `scripts/sql-statements.test.mjs`).	Schema/script safety tests passed locally; no live data-integrity, duplicate/concurrency, or backup/restore test was recorded. **Partial.**	Have an authorized operator run suitable checks on a test database and retain sanitized results. No migration was run for this handover.

#	Evaluation criterion and requirement	Implemented feature / design	Test and source references	Available evidence and status	Outstanding work / limitations
10	**Security and privacy** — verify role-based access and user-data protection.	Role guards distinguish administrator and approved organisation access; response helpers omit selected payment identifiers; privacy policy is present.	auth guards (source path: `lib/auth.ts`), organisation response tests (source path: `lib/organization-request.test.mjs`), cancellation response test (source path: `lib/cancellation-response.test.mjs`), payment response tests (source path: `lib/setup-card-response.test.mjs`), privacy page (source path: `app/privacy/page.tsx`).	Focused privacy/payment-response tests passed locally. **Partial.**	Perform authorized role-by-role access checks and a documented security/privacy review. No penetration test or deployed access test is evidenced.
11	**Client acceptance** — record review against agreed scope.	No acceptance workflow or approval record is in the reviewed repository.	Acceptance record.	No client/mentor acceptance or feedback evidence found. **Not tested / pending acceptance.**	Conduct the review, record reviewer/date/outcomes, and obtain client/mentor confirmation without pre-filling results.

Evidence references

- Evaluation evidence index and automated test inventory
- Automated test report
- Manual testing checklist
- Presentation runbook
- Available evidence and acceptance record

No application screenshots, UML diagrams, executed manual-test records, client feedback, signed acceptance, defect register, or live database-quality results were found. See the detailed evidence index for limitations and the automated test execution record.